Python - Numbers

Python has built-in support to store and process numeric data (**Python Numbers**). Most of the times you work with numbers in almost every Python application. Obviously, any computer application deals with numbers. This tutorial will discuss about different types of Python Numbers and their properties.

Python - Number Types

There are three built-in number types available in Python:

- integers **(int)**
- floating point numbers **(float)**
- **complex** numbers

Python also has a bult-in Boolean data type called **bool**. It can be treated as a sub-type of **int** type, since it's two possible values **True** and **False** represent the integers 1 and 0 respectively.

Python − Integer Numbers

In Python, any number without the provision to store a fractional part is an integer. (Note that if the fractional part in a number is 0, it doesn't mean that it is an integer. For example a number 10.0 is not an integer, it is a float with 0 fractional part whose numeric value is 10.) An integer can be zero, positive or a negative whole number. For example, 1234, 0, -55 all represent to integers in Python.

There are three ways to form an integer object. With (a) literal representation, (b) any expression evaluating to an integer, and (c) using **int()** function.

Literal is a notation used to represent a constant directly in the source code. For example −

```
>>> a =10
```

However, look at the following assignment of the integer variable c.

```
a = 10
b = 20
c = a + b

print ("a:", a, "type:", type(a))
print ("c:", c, "type:", type(c))
```

It will produce the following **output** −

```
a: 10 type: <class 'int'>
c: 30 type: <class 'int'>
```

Here, c is indeed an integer variable, but the expression a + b is evaluated first, and its value is indirectly assigned to c.

The third method of forming an integer object is with the return value of int() function. It converts a floating point number or a string in an integer.

```
>>> a=int(10.5)
>>> b=int("100")
```

You can represent an integer as a binary, octal or Hexa-decimal number. However, internally the object is stored as an integer.

Binary Numbers in Python

A number consisting of only the binary digits (1 and 0) and prefixed with **"0b"** is a binary number. If you assign a binary number to a variable, it still is an int variable.

A represent an integer in binary form, store it directly as a literal, or use int() function, in which the base is set to 2

```
```

```
a=0b101
print ("a:",a, "type:",type(a))

b=int("0b101011", 2)
print ("b:",b, "type:",type(b))
```

It will produce the following **output** –

```
a: 5 type: <class 'int'>
b: 43 type: <class 'int'>
```

There is also a **bin()** function in Python. It returns a binary string equivalent of an integer.

```
a=43
b=bin(a)
print ("Integer:",a, "Binary equivalent:",b)
```

It will produce the following **output** –

```
Integer: 43 Binary equivalent: 0b101011
```

Octal Numbers in Python

An octal number is made up of digits 0 to 7 only. In order to specify that the integer uses octal notation, it needs to be prefixed by **"0o"** (lowercase O) or **"0O"** (uppercase O). A literal representation of octal number is as follows –

```
a=0O107
print (a, type(a))
```

It will produce the following **output** –

```
71 <class 'int'>
```

Note that the object is internally stored as integer. Decimal equivalent of octal number 107 is 71.

Since octal number system has 8 symbols (0 to 7), its base is 7. Hence, while using int() function to covert an octal string to integer, you need to set the base argument to 8.

```
a=int('20',8)
print (a, type(a))
```

It will produce the following **output** –

```
16 <class 'int'>
```

Decimal equivalent of octal 30 is 16.

In the following code, two int objects are obtained from octal notations and their addition is performed.

```
a=0056
print ("a:",a, "type:",type(a))

b=int("0031",8)
print ("b:",b, "type:",type(b))

c=a+b
print ("addition:", c)
```

It will produce the following **output** –

```
a: 46 type: <class 'int'>
b: 25 type: <class 'int'>
addition: 71
```

To obtain the octal string for an integer, use **oct()** function.

```
a=oct(71)
print (a, type(a))
```

Hexa-decimal Numbers in Python

As the name suggests, there are 16 symbols in the Hexadecimal number system. They are 0-9 and A to F. The first 10 digits are same as decimal digits. The alphabets A, B, C, D, E and F are equivalents of 11, 12, 13, 14, 15, and 16 respectively. Upper or lower cases may be used for these letter symbols.

For the literal representation of an integer in Hexadecimal notation, prefix it by **"0x"** or **"0X"**.

```
a=0XA2
print (a, type(a))
```

It will produce the following **output** –

```
162 <class 'int'>
```

To convert a Hexadecimal string to integer, set the base to 16 in the **int()** function.

```
a=int('0X1e', 16)
print (a, type(a))
```

Try out the following code snippet. It takes a Hexadecimal string, and returns the integer.

```
```

```
num_string = "A1"
number = int(num_string, 16)
print ("Hexadecimal:", num_string, "Integer:",number)
```

It will produce the following **output** –

```
Hexadecimal: A1 Integer: 161
```

However, if the string contains any symbol apart from the Hexadecimal symbol chart an error will be generated.

```
num_string = "A1X001"
print (int(num_string, 16))
```

The above program generates the following error –

```
Traceback (most recent call last):
  File "/home/main.py", line 2, in
    print (int(num_string, 16))
ValueError: invalid literal for int() with base 16: 'A1X001'
```

Python's standard library has **hex()** function, with which you can obtain a hexadecimal equivalent of an integer.

```
a=hex(161)
print (a, type(a))
```

It will produce the following **output** –

```
0xa1 <class 'str'>
```

Though an integer can be represented as binary or octal or hexadecimal, internally it is still integer. So, when performing arithmetic operation, the representation doesn't

matter.

```
a=10 #decimal
b=0b10 #binary
c=0O10 #octal
d=0XA #Hexadecimal
e=a+b+c+d

print ("addition:", e)
```

It will produce the following **output** –

```
addition: 30
```

Python − Floating Point Numbers

A floating point number has an integer part and a fractional part, separated by a decimal point symbol (.). By default, the number is positive, prefix a dash (-) symbol for a negative number.

A floating point number is an object of Python's float class. To store a float object, you may use a literal notation, use the value of an arithmetic expression, or use the return value of float() function.

Using literal is the most direct way. Just assign a number with fractional part to a variable. Each of the following statements declares a float object.

```
>>> a=9.99
>>> b=0.999
>>> c=-9.99
>>> d=-0.999
```

In Python, there is no restriction on how many digits after the decimal point can a floating point number have. However, to shorten the representation, the **E** or **e** symbol is used. E stands for Ten raised to. For example, E4 is 10 raised to 4 (or 4^{th} power of 10), e-3 is 10 raised to -3.

In scientific notation, number has a coefficient and exponent part. The coefficient should be a float greater than or equal to 1 but less than 10. Hence, 1.23E+3, 9.9E-5, and 1E10 are the examples of floats with scientific notation.

```
>>> a=1E10
>>> a
10000000000.0
>>> b=9.90E-5
>>> b
9.9e-05
>>> 1.23E3
1230.0
```

The second approach of forming a float object is indirect, using the result of an expression. Here, the quotient of two floats is assigned to a variable, which refers to a float object.

```
a=10.33
b=2.66
c=a/b

print ("c:", c, "type", type(c))
```

It will produce the following **output** –

```
c: 3.8834586466165413 type <class 'float'>
```

Python's float() function returns a float object, parsing a number or a string if it has the appropriate contents. If no arguments are given in the parenthesis, it returns 0.0, and for an **int** argument, fractional part with 0 is added.

```
>>> a=float()
>>> a
0.0
>>> a=float(10)
>>> a
10.0
```

Even if the integer is expressed in binary, octal or hexadecimal, the float() function returns a float with fractional part as 0.

```
a=float(0b10)
b=float(0O10)
c=float(0xA)

print (a,b,c, sep=",")
```

It will produce the following **output** –

```
2.0,8.0,10.0
```

The **float()** function retrieves a floating point number out of a string that encloses a float, either in standard decimal point format, or having scientific notation.

```
a=float("-123.54")
b=float("1.23E04")
print ("a=",a,"b=",b)
```

It will produce the following **output** –

```
a= -123.54 b= 12300.0
```

In mathematics, infinity is an abstract concept. Physically, infinitely large number can never be stored in any amount of memory. For most of the computer hardware configurations, however, a very large number with 400th power of 10 is represented by Inf. If you use "Infinity" as argument for float() function, it returns Inf.

```
a=1.00E400
print (a, type(a))
a=float("Infinity")
print (a, type(a))
```

It will produce the following **output** –

```
inf <class 'float'>
inf <class 'float'>
```

One more such entity is Nan (stands for Not a Number). It represents any value that is undefined or not representable.

```
>>> a=float('Nan')
>>> a
Nan
```

Python − Complex Numbers

In this section, we shall know in detail about Complex data type in Python. Complex numbers find their applications in mathematical equations and laws in electromagnetism, electronics, optics, and quantum theory. Fourier transforms use complex numbers. They are Used in calculations with wavefunctions, designing filters, signal integrity in digital electronics, radio astronomy, etc.

A complex number consists of a real part and an imaginary part, separated by either "+" or "−". The real part can be any floating point (or itself a complex number) number. The imaginary part is also a float/complex, but multiplied by an imaginary number.

In mathematics, an imaginary number "i" is defined as the square root of -1 (\sqrt{−1}). Therefore, a complex number is represented as "x+yi", where x is the real part, and "y" is the coefficient of imaginary part.

Quite often, the symbol "j" is used instead of "I" for the imaginary number, to avoid confusion with its usage as current in theory of electricity. Python also uses "j" as the imaginary number. Hence, "x+yj" is the representation of complex number in Python.

Like int or float data type, a complex object can be formed with literal representation or using complex() function. All the following statements form a complex object.

```
>>> a=5+6j
>>> a
(5+6j)
>>> type(a)
<class 'complex'>
>>> a=2.25-1.2J
>>> a
(2.25-1.2j)
>>> type(a)
<class 'complex'>
```

```
>>> a=1.01E-2+2.2e3j
>>> a
(0.0101+2200j)
>>> type(a)
<class 'complex'>
```

Note that the real part as well as the coefficient of imaginary part have to be floats, and they may be expressed in standard decimal point notation or scientific notation.

Python's **complex()** function helps in forming an object of complex type. The function receives arguments for real and imaginary part, and returns the complex number.

There are two versions of complex() function, with two arguments and with one argument. Use of complex() with two arguments is straightforward. It uses first argument as real part and second as coefficient of imaginary part.

```
a=complex(5.3,6)
b=complex(1.01E-2, 2.2E3)
print ("a:", a, "type:", type(a))
print ("b:", b, "type:", type(b))
```

It will produce the following **output** –

```
a: (5.3+6j) type: <class 'complex'>
b: (0.0101+2200j) type: <class 'complex'>
```

In the above example, we have used x and y as float parameters. They can even be of complex data type.

```
a=complex(1+2j, 2-3j)
print (a, type(a))
```

It will produce the following **output** –

```
(4+4j) <class 'complex'>
```

Surprised by the above example? Put "x" as 1+2j and "y" as 2-3j. Try to perform manual computation of "x+yj" and you'll come to know.

```
complex(1+2j, 2-3j)
=(1+2j)+(2-3j)*j
=1+2j +2j+3
=4+4j
```

If you use only one numeric argument for complex() function, it treats it as the value of real part; and imaginary part is set to 0.

```
a=complex(5.3)
print ("a:", a, "type:", type(a))
```

It will produce the following **output** –

```
a: (5.3+0j) type: <class 'complex'>
```

The complex() function can also parse a string into a complex number if its only argument is a string having complex number representation.

In the following snippet, user is asked to input a complex number. It is used as argument. Since Python reads the input as a string, the function extracts the complex object from it.

```
a= "5.5+2.3j"
b=complex(a)
print ("Complex number:", b)
```

It will produce the following **output** –

```
Complex number: (5.5+2.3j)
```

Python's built-in complex class has two attributes **real** and **imag** – they return the real and coefficient of imaginary part from the object.

```
a=5+6j
print ("Real part:", a.real, "Coefficient of Imaginary part:", a.imag)
```

It will produce the following **output** –

```
Real part: 5.0 Coefficient of Imaginary part: 6.0
```

The complex class also defines a conjugate() method. It returns another complex number with the sign of imaginary component reversed. For example, conjugate of x+yj is x-yj.

```
>>> a=5-2.2j
>>> a.conjugate()
(5+2.2j)
```

Number Type Conversion

Python converts numbers internally in an expression containing mixed types to a common type for evaluation. But sometimes, you need to coerce a number explicitly from one type to another to satisfy the requirements of an operator or function parameter.

- Type int(x) to convert x to a plain integer.
- Type long(x) to convert x to a long integer.
- Type float(x) to convert x to a floating-point number.
- Type complex(x) to convert x to a complex number with real part x and imaginary part zero. In the same way type **complex(x, y)** to convert x and y to a complex number with real part x and imaginary part y. x and y are numeric expressions

Let us see various numeric and math-related functions.

Theoretic and Representation Functions

Python includes following theoretic and representation Functions in the **math** module –

Sr.No.	Function & Description
1	**math.ceil(x)** The ceiling of x: the smallest integer not less than x
2	**math.comb(n,k)** This function is used to find the returns the number of ways to choose "x" items from "y" items without repetition and without order.
3	**math.copysign(x, y)** This function returns a float with the magnitude (absolute value) of x but the sign of y.
4	**math.cmp(x, y)** This function is used to compare the values of to objects. This function is deprecated in Python3.
5	**math.fabs(x)** This function is used to calculate the absolute value of a given integer.
6	**math.factorial(n)** This function is used to find the factorial of a given integer.
7	**math.floor(x)** This function calculates the floor value of a given integer.
8	**math.fmod(x, y)** The fmod() function in math module returns same result as the **"%"** operator. However fmod() gives more accurate result of modulo division than modulo operator.
9	**math.frexp(x)** This function is used to calculate the mantissa and exponent of a given number.
10	**math.fsum(iterable)** This function returns the floating point sum of all numeric items in an iterable i.e. list, tuple, array.
11	**math.gcd(*integers)** This function is used to calculate the greatest common divisor of all the given integers.
12	**math.isclose()** This function is used to determine whether two given numeric values are close to each other.
13	**math.isfinite(x)**

	This function is used to determine whether the given number is a finite number.
14	**math.isinf(x)** This function is used to determine whether the given value is infinity (+ve or, -ve).
15	**math.isnan(x)** This function is used to determine whether the given number is "NaN".
16	**math.isqrt(n)** This function calculates the integer square-root of the given non negative integer.
17	**math.lcm(*integers)** This function is used to calculate the least common factor of the given integer arguments.
18	**math.ldexp(x, i)** This function returns product of first number with exponent of second number. So, ldexp(x,y) returns x*2**y. This is inverse of frexp() function.
19	**math.modf(x)** This returns the fractional and integer parts of x in a two-item tuple.
20	**math.nextafter(x, y, steps)** This function returns the next floating-point value after x towards y.
21	**math.perm(n, k)** This function is used to calculate the permutation. It returns the number of ways to choose x items from y items without repetition and with order.
22	**math.prod(iterable, *, start)** This function is used to calculate the product of all numeric items in the iterable (list, tuple) given as argument.
23	**math.remainder(x,y)** This function returns the remainder of x with respect to y. This is the difference x − n*y, where n is the integer closest to the quotient x / y.
24	**math.trunc(x)** This function returns integral part of the number, removing the fractional part. trunc() is equivalent to floor() for positive x, and equivalent to ceil() for negative x.
25	**math.ulp(x)** This function returns the value of the least significant bit of the float x. trunc() is equivalent to floor() for positive x, and equivalent to ceil() for

	negative x.

Power and Logarithmic Functions

Sr.No.	Function & Description
1	**math.cbrt(x)** This function is used to calculate the cube root of a number.
2	**math.exp(x)** This function calculate the exponential of x: e^x
3	**math.exp2(x)** This function returns 2 raised to power x. It is equivalent to 2**x.
4	**math.expm1(x)** This function returns e raised to the power x, minus 1. Here e is the base of natural logarithms.
5	**math.log(x)** This function calculates the natural logarithm of x, for x> 0.
6	**math.log1p(x)** This function returns the natural logarithm of 1+x (base e). The result is calculated in a way which is accurate for x near zero.
7	**math.log2(x)** This function returns the base-2 logarithm of x. This is usually more accurate than log(x, 2).
8	**math.log10(x)** The base-10 logarithm of x for x> 0.
9	**math.pow(x, y)** The value of x**y.
10	**math.sqrt(x)** The square root of x for x > 0

Trigonometric Functions

Python includes following functions that perform trigonometric calculations in the **math** module –

Sr.No.	Function & Description

1	**math.acos(x)** This function returns the arc cosine of x, in radians.
2	**math.asin(x)** This function returns the arc sine of x, in radians.
3	**math.atan(x)** This function returns the arc tangent of x, in radians.
4	**math.atan2(y, x)** This function returns atan(y / x), in radians.
5	**math.cos(x)** This function returns the cosine of x radians.
6	**math.sin(x)** This function returns the sine of x radians.
7	**math.tan(x)** This function returns the tangent of x radians.
8	**math.hypot(x, y)** This function returns the Euclidean norm, sqrt(x*x + y*y).

Angular conversion Functions

Following are the angular conversion function provided by Python **math** module –

Sr.No.	Function & Description
1	**math.degrees(x)** This function converts the given angle from radians to degrees.
2	**math.radians(x)** This function converts the given angle from degrees to radians.

Mathematical Constants

The Python **math** module defines the following mathematical constants –

Sr.No.	Constants & Description
1	**math.pi** This represents the mathematical constant pi, which equals to **"3.141592..."** to available precision.

2	**math.e** This represents the mathematical constant e, which is equal to **"2.718281..."** to available precision.
3	**math.tau** This represents the mathematical constant Tau (denoted by). It is equivalent to the ratio of circumference to radius, and is equal to **2**.
4	**math.inf** This represents positive infinity. For negative infinity use **"−math.inf"**.
5	**math.nan** This constant is a floating-point "not a number" (NaN) value. Its value is equivalent to the output of float('nan').

Hyperbolic Functions

Hyperbolic functions are analogs of trigonometric functions that are based on hyperbolas instead of circles. Following are the hyperbolic functions of the Python **math** module −

Sr.No.	Function & Description
1	**math.acosh(x)** This function is used to calculate the inverse hyperbolic cosine of the given value.
2	**math.asinh(x)** This function is used to calculate the inverse hyperbolic sine of a given number.
3	**math.atanh(x)** This function is used to calculate the inverse hyperbolic tangent of a number.
4	**math.cosh(x)** This function is used to calculate the hyperbolic cosine of the given value.
5	**math.sinh(x)** This function is used to calculate the hyperbolic sine of a given number.
6	**math.tanh(x)** This function is used to calculate the hyperbolic tangent of a number.

Special Functions

Following are the special functions provided by the Python **math** module −

Sr.No.	Function & Description
1	**math.erf(x)** This function returns the value of the Gauss error function for the given parameter.
2	**math.erfc(x)** This function is the complementary for the error function. Value of erf(x) is equivalent to **1-erf(x)**.
3	**math.gamma(x)** This is used to calculate the factorial of the complex numbers. It is defined for all the complex numbers except the non-positive integers.
4	**math.lgamma(x)** This function is used to calculate the natural logarithm of the absolute value of the Gamma function at x.

Random Number Functions

Random numbers are used for games, simulations, testing, security, and privacy applications. Python includes following functions in the **random** module.

Sr.No.	Function & Description
1	**random.choice(seq)** A random item from a list, tuple, or string.
2	**random.randrange([start,] stop [,step])** A randomly selected element from range(start, stop, step)
3	**random.random()** A random float r, such that 0 is less than or equal to r and r is less than 1
4	**random.seed([x])** This function sets the integer starting value used in generating random numbers. Call this function before calling any other random module function. Returns None.
5	**random.shuffle(seq)** This function is used to randomize the items of the given sequence.
6	**random.uniform(a, b)** This function returns a random floating point value r, such that a is less than or equal to r and r is less than b.

Built-in Mathematical Functions

Following mathematical functions are built into the Python interpreter, hence you don't need to import them from any module.

Sr.No.	Function & Description
1	**Python abs() function** The abs() function returns the absolute value of x, i.e. the positive distance between x and zero.
2	**Python max() function** The max() function returns the largest of its arguments or largest number from the iterable (list or tuple).
3	**Python min() function** The function min() returns the smallest of its arguments i.e. the value closest to negative infinity, or smallest number from the iterable (list or tuple)
4	**Python pow() function** The pow() function returns x raised to y. It is equivalent to x**y.
5	**Python round() Function** round() is a built-in function in Python. It returns x rounded to n digits from the decimal point.
6	**Python sum() function** The sum() function returns the sum of all numeric items in any iterable (list or tuple). It has an optional start argument which is 0 by default. If given, the numbers in the list are added to start value.

JavaScript Tutorial

SQL Tutorial

TRENDING TECHNOLOGIES

Cloud Computing Tutorial

Amazon Web Services Tutorial

Microsoft Azure Tutorial

Git Tutorial

Ethical Hacking Tutorial

Docker Tutorial

Kubernetes Tutorial

DSA Tutorial

Spring Boot Tutorial

SDLC Tutorial

Unix Tutorial

CERTIFICATIONS

Business Analytics Certification

Java & Spring Boot Advanced Certification

Data Science Advanced Certification

Cloud Computing And DevOps

Advanced Certification In Business Analytics

Artificial Intelligence And Machine Learning

DevOps Certification

Game Development Certification

Front-End Developer Certification

AWS Certification Training

Python Programming Certification

COMPILERS & EDITORS

Online Java Compiler

Online Python Compiler

Online Go Compiler

Online C Compiler

Online C++ Compiler